FATIMA JINNAH WOMEN UNIVERSITY

Rawalpindi

DevOps – Lab 3

Working with Git History, Stashing, and Reverting Commits

Name	Mahnoor Sajjad
Roll No.	2023-BCS-054 (Section A)
Program	BS Computer Science
Course	DevOps
Lab	Lab 3 (Git History, Stashing & Reverting)
GitHub Username	mahnoor-cs6767
Repository	https://github.com/mahnoor-cs6767/DevOps

Objective

In this lab I practised the Git commands that are used to look back at history, fix mistakes and keep a shared repository clean. I did everything in my own repository, DevOps, under the folder Labs/Lab03, and took a screenshot after each important step as evidence.

The things I covered in this lab were:

- Handling local and remote commit conflicts with git pull and git pull --rebase
- Resolving merge conflicts manually
- Using .gitignore and removing already tracked files with git rm --cached
- Saving unfinished work temporarily with git stash
- Checking out an older commit using its hash (detached HEAD)
- Soft reset vs hard reset
- Amending the last commit
- Reverting a commit safely on a remote branch
- Force pushing on a personal branch
- Running Gitea in GitHub Codespaces with Docker Compose
- Publishing a portfolio site with GitHub Pages

Environment

- Local machine: Ubuntu virtual machine (VMware), working inside the folder ~/lab1-submission
- Editor: nano in the terminal
- Remote: GitHub repository https://github.com/mahnoor-cs6767/DevOps (connected over HTTPS)
- Tasks 10 and 11 were done in GitHub Codespaces and VS Code

Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

The aim of this task was to see what happens when both the remote and my local repository have a new commit, and to compare the two ways of fixing it: git pull (merge) and git pull --rebase.

Part A: Merge

1. I opened the repository on GitHub, edited Labs/Lab03/README.md in the browser, added the line “This line was added remotely from GitHub” and committed it.

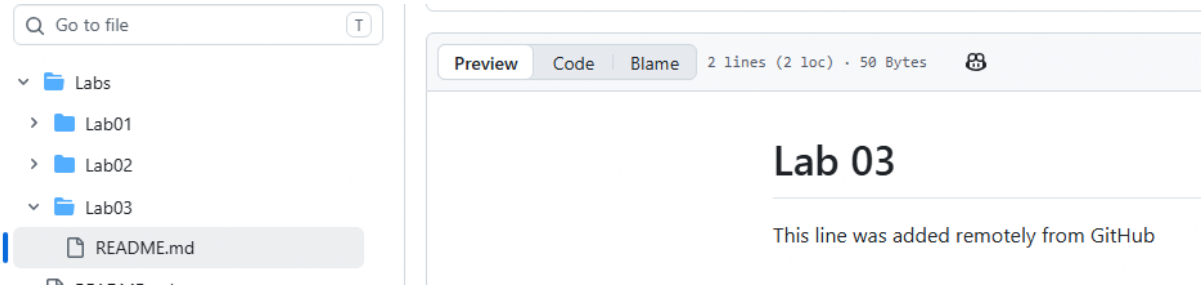


Figure 1: README.md edited and committed directly on GitHub

2. On my machine I added a different line to the same README.md, staged it and committed it.

```
echo "This line was added locally." >> Labs/Lab03/README.md
git add Labs/Lab03/README.md
git commit -m "Local update to README"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "This line was added locally." >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Local update to README"
[main 5aa0204] Local update to README
 1 file changed, 2 insertions(+), 1 deletion(-)
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$
```

Figure 2: Local commit “Local update to README” (5aa0204)

3. I tried to push with `git push origin main`. GitHub rejected it with “fetch first” because the remote already had a commit that I did not have locally.

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
To https://github.com/mahnoor-cs6767/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/mahnoor-cs6767/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$
```

Figure 3: Push rejected because the remote contains work I do not have locally

4. I pulled the remote changes with a merge. Both of us had edited the README, so I opened it in nano, cleaned up the file, then staged it and committed the result.

```
git pull --no-rebase origin main
nano Labs/Lab03/README.md
git add Labs/Lab03/README.md
git commit -m "Resolved merge conflict in README.md"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ nano Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Resolved merge conflict in README.md"
[main 0c0e72b] Resolved merge conflict in README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$
```

Figure 4: Merge resolved in nano and committed as 0c0e72b

5. After that the push went through.

```
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 982 bytes | 122.00 KiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/mahnoor-cs6767/DevOps.git
   6bbba2a..0c0e72b  main -> main
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$
```

Figure 5: Push succeeded after the merge (6bbba2a..0c0e72b)

Part B: Rebase

6. For the second round I used `git pull --rebase origin main` instead. Rebase takes my local commits and replays them on top of the commits pulled from the remote, so the history stays in one straight line.

```
git pull --rebase origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git pull --rebase origin main
From https://github.com/mahnoor-cs6767/DevOps
 * branch            main       -> FETCH_HEAD
Already up to date.
```

Figure 6: git pull --rebase origin main

7. Git reported that everything was already up to date, so there was nothing new to replay. I then pushed.

```
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Everything up-to-date
```

Figure 7: Push after the rebase pull

What I learned

- git pull --no-rebase merges the remote work with mine and can create an extra merge commit.
- git pull --rebase puts my commits on top of the remote ones, so the history stays linear.
- Both are valid ways to fix a rejected push when my local branch is behind the remote.

Task 2 – Creating and Resolving Merge Conflicts Manually

This time both sides changed the same line, so Git could not decide on its own which version to keep. I had to resolve the conflict by hand.

1. On GitHub I changed the existing line in README.md to “This line was updated remotely again.” and committed it with the message Remote conflicting change.

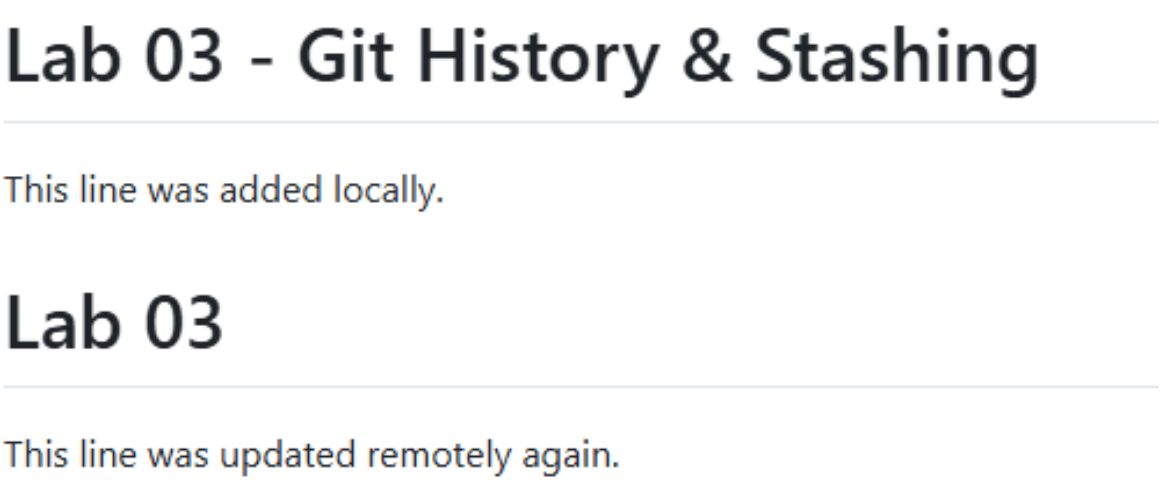


Figure 8: Remote change to the same line, committed on GitHub

2. Locally I changed the same line in nano to “This line was updated locally at the same time.”

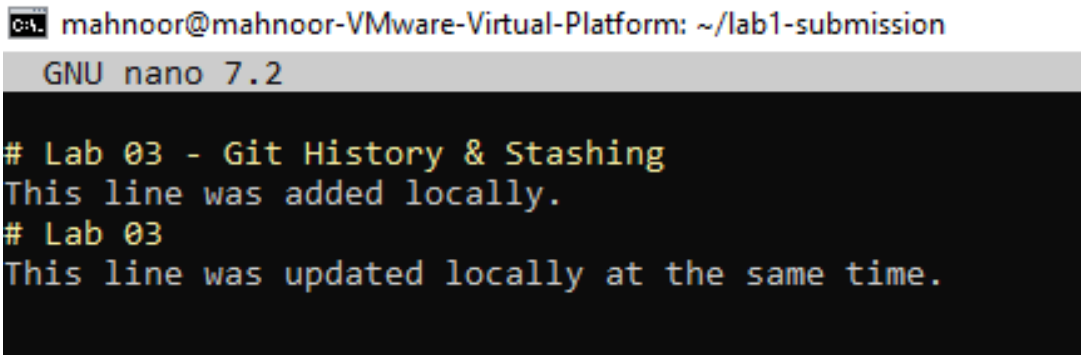


Figure 9: Local edit of the same line in nano

3. I staged and committed the local change.

```
git add Labs/Lab03/README.md
git commit -m "Local conflicting change"
```

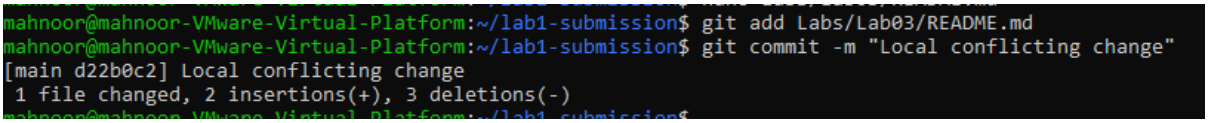


Figure 10: Local commit “Local conflicting change” (d22b0c2)

4. I tried to push and it was rejected, as expected.

```
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
To https://github.com/mahnoor-cs6767/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/mahnoor-cs6767/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Figure 11: Push rejected because the remote has a conflicting commit

5. I pulled with rebase. Git stopped and reported "CONFLICT (content): Merge conflict in Labs/Lab03/README.md" and told me to fix the file, run `git add` and then `git rebase --continue`.

```
git pull --rebase origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git pull --rebase origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 1.10 KiB | 140.00 KiB/s, done.
From https://github.com/mahnoor-cs6767/DevOps
 * branch            main       -> FETCH_HEAD
   0c0e72b..19f2c16  main       -> origin/main
Auto-merging Labs/Lab03/README.md
CONFLICT (content): Merge conflict in Labs/Lab03/README.md
error: could not apply d22b0c2... Local conflicting change
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
Could not apply d22b0c2... Local conflicting change
```

Figure 12: Rebase paused with a content conflict in README.md

6. I opened README.md in nano, where Git had added the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`). I removed the markers and kept the lines I wanted.

```
nano Labs/Lab03/README.md
```

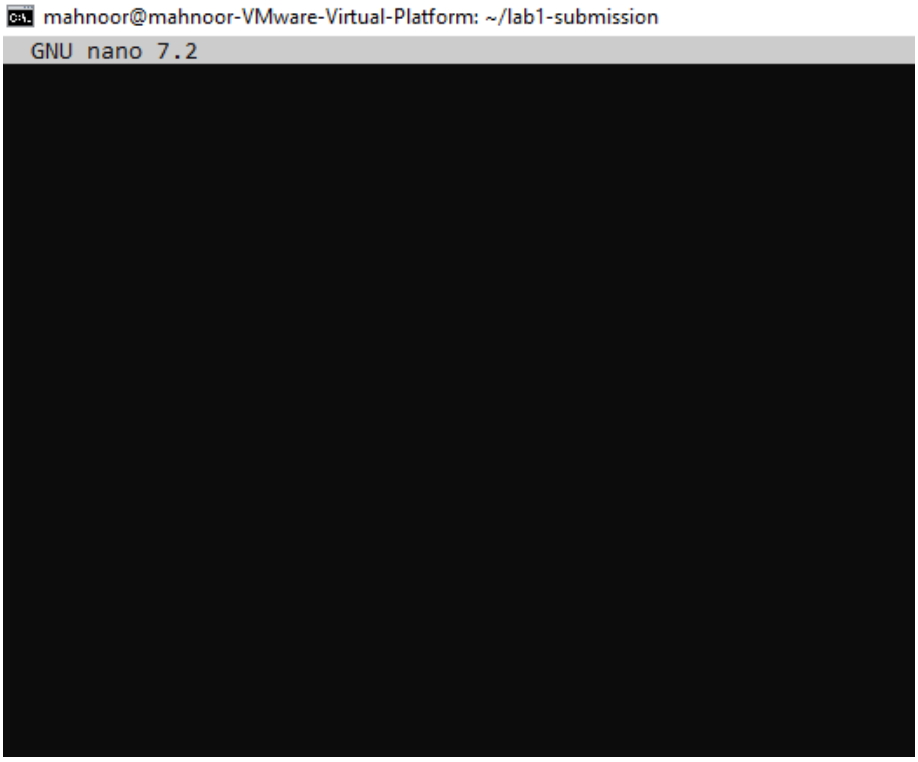


Figure 13: README.md open in nano while resolving the conflict

7. I marked the file as resolved and continued the rebase. Git opened the commit message editor, which I saved as it was.

```
git add Labs/Lab03/README.md
git rebase --continue
```

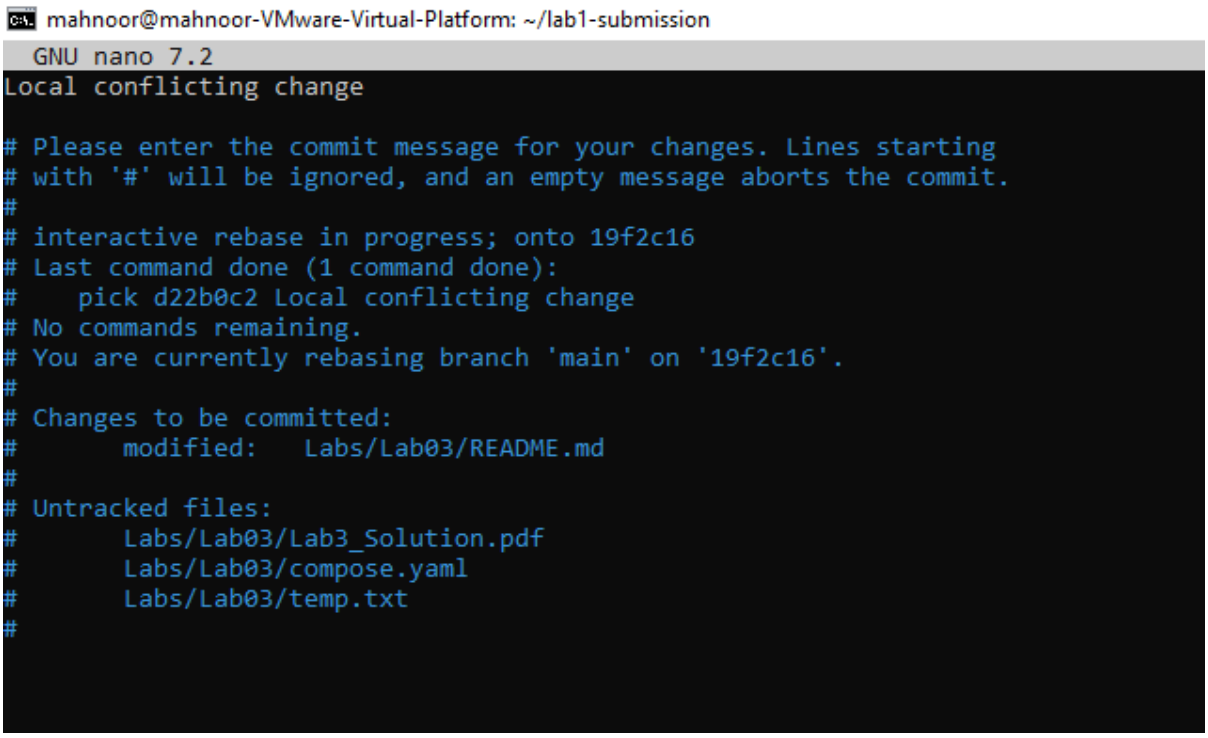


Figure 14: Continuing the rebase (commit message "Local conflicting change")

8. Finally I pushed the result to GitHub.

```
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 397 bytes | 198.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mahnoor-cs6767/DevOps.git
   19f2c16..d38a63d  main -> main
```

Figure 15: Push succeeded after resolving the conflict (19f2c16..d38a63d)

What I learned

- When two people change the same line, Git pauses and asks me to choose what to keep.
- The conflict markers show my version (HEAD) and the remote version, and I must edit the file myself.
- After fixing the file I use git add and git rebase --continue, then push.

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

Here I learned that .gitignore only stops new files from being tracked. Files that are already committed stay on the remote until they are removed from tracking.

1. I created a folder called textfiles inside Labs/Lab03 with three files, then added, committed and pushed them.

```
mkdir -p Labs/Lab03/textfiles
echo "File A content" > Labs/Lab03/textfiles/a.txt
echo "File B content" > Labs/Lab03/textfiles/b.txt
echo "File C content" > Labs/Lab03/textfiles/c.txt
git add .
git commit -m "Added textfiles directory with three files"
git push origin main
```

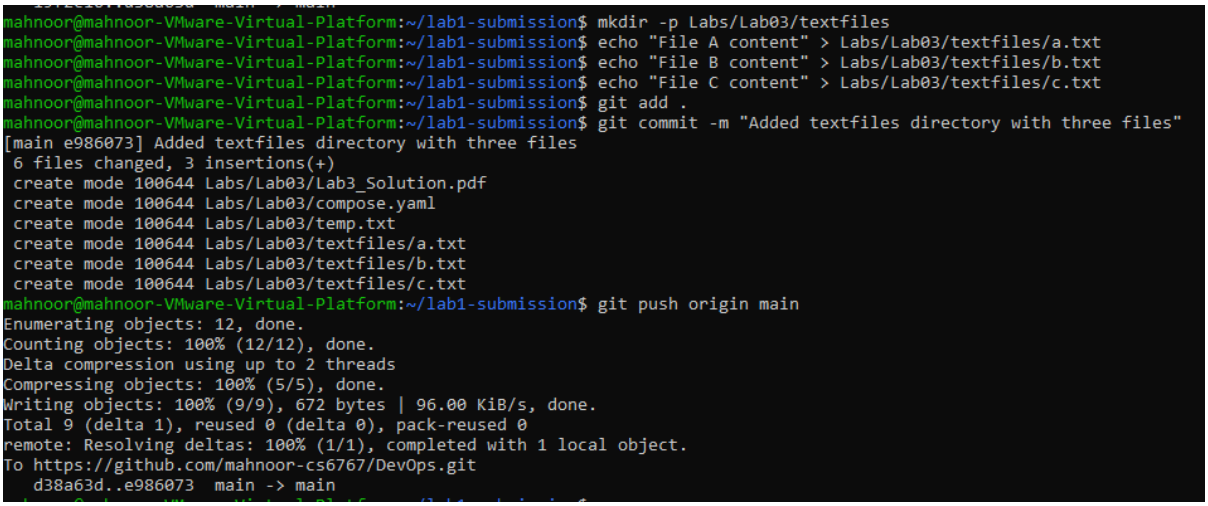


Figure 16: textfiles directory with a.txt, b.txt and c.txt committed and pushed

2. I created a .gitignore file for that folder, committed it and pushed.

```
echo "Labs/Lab03/textfiles/*" >> .gitignore
git add .gitignore
git commit -m "Added .gitignore to ignore textfiles directory"
git push origin main
```

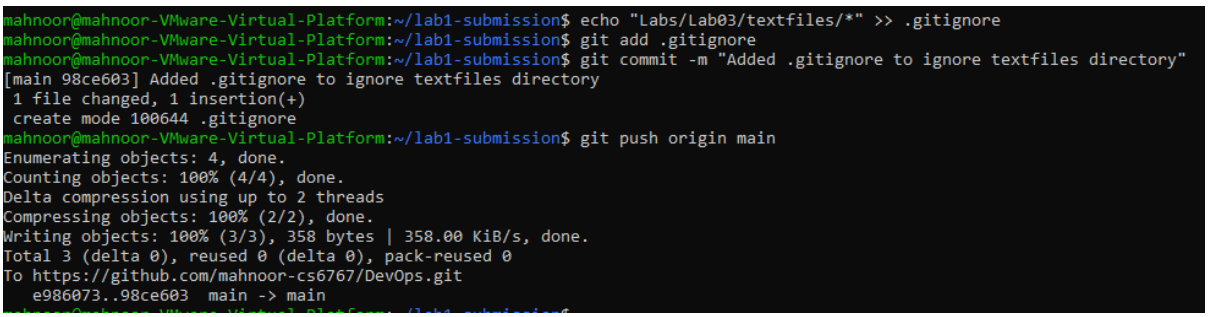


Figure 17: .gitignore added and pushed

3. On GitHub I checked the repository. The textfiles folder with a.txt, b.txt and c.txt was still there, because they had been committed before the ignore rule existed.

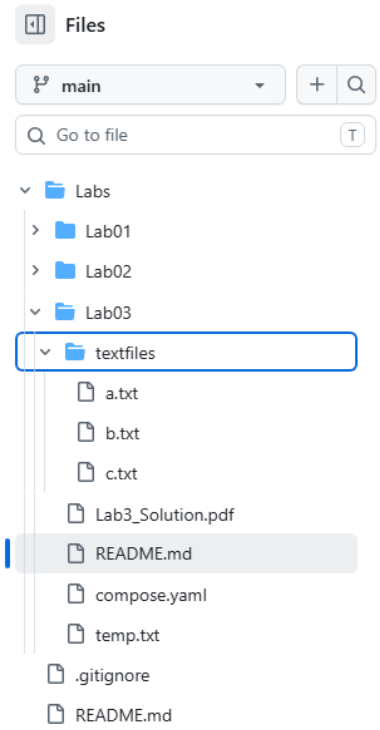


Figure 18: textfiles is still visible on GitHub after adding .gitignore

4. To untrack the folder but keep it on my computer I used `git rm -r --cached`, then committed and pushed.

```
git rm -r --cached Labs/Lab03/textfiles
git add .
git commit -m "Removed tracked textfiles directory"
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git rm -r --cached Labs/Lab03/textfiles
rm 'Labs/Lab03/textfiles/a.txt'
rm 'Labs/Lab03/textfiles/b.txt'
rm 'Labs/Lab03/textfiles/c.txt'
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Removed tracked textfiles directory"
[main 9df4dad] Removed tracked textfiles directory
 3 files changed, 3 deletions(-)
 delete mode 100644 Labs/Lab03/textfiles/a.txt
 delete mode 100644 Labs/Lab03/textfiles/b.txt
 delete mode 100644 Labs/Lab03/textfiles/c.txt
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 411 bytes | 205.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/mahnoor-cs6767/DevOps.git
   98ce603..9df4dad  main -> main
```

Figure 19: Files removed from tracking and the change pushed (98ce603..9df4dad)

5. I checked GitHub again and the textfiles folder was gone from the remote.

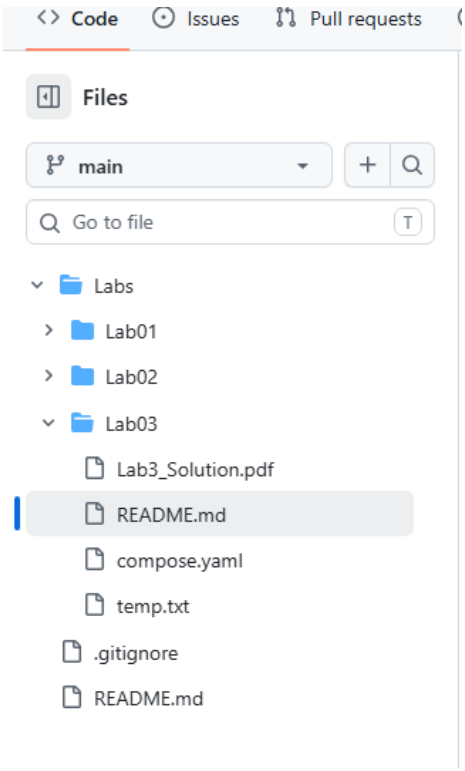


Figure 20: textfiles folder no longer present on GitHub

What I learned

- .gitignore does not affect files that are already tracked.
- git rm --cached removes a file from Git without deleting it from my disk.
- After committing and pushing that change, the files disappear from the remote repository.

Task 4 – Create Temporary Changes and Use git stash

git stash lets me put away uncommitted work so that I can switch branches without losing anything.

1. I created a new branch called feature-branch, added a line to README.md and committed it.

```
git checkout -b feature-branch
echo "Feature changes" >> Labs/Lab03/README.md
git add .
git commit -m "Changes the README.md file"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout -b feature-branch
Switched to a new branch 'feature-branch'
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Feature changes" >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Changes the README.md file"
[feature-branch f94a66f] Changes the README.md file
 1 file changed, 1 insertion(+)
```

Figure 21: New branch feature-branch with a committed README change (f94a66f)

2. I made another change to README.md but did not commit it, and tried to switch to main. Git refused because the checkout would overwrite my local changes.

```
echo "Uncommitted changes" >> Labs/Lab03/README.md
git checkout main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Uncommitted changes" >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
        Labs/Lab03/README.md
Please commit your changes or stash them before you switch branches.
Aborting
```

Figure 22: Checkout blocked: local changes would be overwritten

3. I stashed my changes.

```
git stash
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git stash
Saved working directory and index state WIP on feature-branch: f94a66f Changes the README.md file
```

Figure 23: Changes saved with git stash

4. With the working directory clean I could switch to main.

```
git checkout main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Figure 24: Switched to main successfully

5. Then I went back to feature-branch.

```
git checkout feature-branch
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout feature-branch
Switched to branch 'feature-branch'
```

Figure 25: Back on feature-branch

6. git status showed that the working tree was clean, because my edit was still inside the stash.

```
git status
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git status
On branch feature-branch
nothing to commit, working tree clean
```

Figure 26: git status: nothing to commit, working tree clean

7. Finally I restored the stashed work.

```
git stash pop
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git stash pop
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Labs/Lab03/README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (3889cd92330bacac3f5d9aeb465cf9f7ad0e406d)
```

Figure 27: git stash pop brings back the modified README.md

What I learned

- git stash saves uncommitted changes so I can switch branches safely.
- git stash pop brings the changes back and removes the stash entry.
- It is a quick way to keep work-in-progress without adding extra commits.

Task 5 – Checkout a Specific Commit Using git log

In this task I went back to an older version of the project by checking out a commit hash.

1. I listed the history in short form.

```
git log --oneline
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git log --oneline
9df4dad (HEAD -> main, origin/main) Removed tracked textfiles directory
98ce603 Added .gitignore to ignore textfiles directory
e986073 Added textfiles directory with three files
d38a63d Local conflicting change
19f2c16 Update README.md
0c0e72b Resolved merge conflict in README.md
5aa0204 Local update to README
6bbba2a Update README.md
d556464 Updated directory structure according to requirements
7fdd998 Added screenshots folder and actual PDF
a3d1e8e Added labs structure and files
296b521 initial commit for lab 01
```

Figure 28: Commit history before checking out an old commit

2. I picked the hash e986073 ("Added textfiles directory with three files") and checked it out. Git warned me that I was now in a detached HEAD state.

```
git checkout e986073
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout e986073
Note: switching to 'e986073'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at e986073 Added textfiles directory with three files
```

Figure 29: Detached HEAD at commit e986073

3. To finish I returned to the main branch.

```
git checkout main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout main
Previous HEAD position was e986073 Added textfiles directory with three files
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Figure 30: Back on main

What I learned

- Checking out a hash lets me look at the project exactly as it was at that commit.

- In detached HEAD, new commits do not belong to any branch, so I should not work there.
- I always go back to main after inspecting old commits.

Task 6 – Resetting Commits (Soft vs Hard Reset)

This task compares git reset --soft and git reset --hard. After each reset I checked both the commit history and the file contents.

1. I added “Test line 1” to README.md and committed it.

```
echo "Test line 1" >> Labs/Lab03/README.md
git add .
git commit -m "Added test line"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Test line 1" >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Added test line"
[main a9386f4] Added test line
 1 file changed, 1 insertion(+)
```

Figure 31: First test commit (a9386f4)

2. I added “Test line 2” and made a second commit.

```
echo "Test line 2" >> Labs/Lab03/README.md
git add .
git commit -m "Second test commit"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Test line 2" >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Second test commit"
[main 3bb08c6] Second test commit
 1 file changed, 1 insertion(+)
```

Figure 32: Second test commit (3bb08c6)

3. I viewed the history before any reset. The two new commits are on top of origin/main.

```
git log --oneline
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git log --oneline
3bb08c6 (HEAD -> main) Second test commit
a9386f4 Added test line
9df4dad (origin/main) Removed tracked textfiles directory
98ce603 Added .gitignore to ignore textfiles directory
e986073 Added textfiles directory with three files
d38a63d Local conflicting change
19f2c16 Update README.md
0c0e72b Resolved merge conflict in README.md
5aa0204 Local update to README
6bbba2a Update README.md
d556464 Updated directory structure according to requirements
7fdd998 Added screenshots folder and actual PDF
a3d1e8e Added labs structure and files
296b521 initial commit for lab 01
```

Figure 33: History before reset

4. I checked the file and both lines were there.

```
cat Labs/Lab03/README.md
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ cat Labs/Lab03/README.md

Test line 1
Test line 2
```

Figure 34: README.md contains Test line 1 and Test line 2

Soft reset

5. I moved back one commit with a soft reset.

```
git reset --soft HEAD~1
```

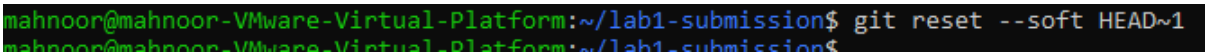


Figure 35: git reset --soft HEAD~1

6. The log now ends at “Added test line”, so the second commit is gone from history.

```
git log --oneline
```

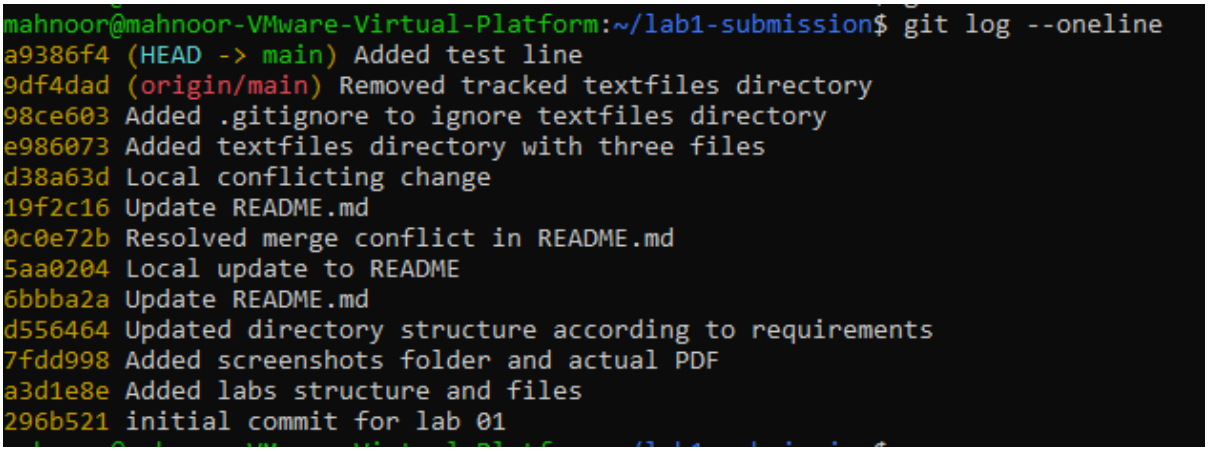


Figure 36: History after soft reset

7. The file still has both lines, so nothing was lost. The changes stay staged and ready to commit again.

```
cat Labs/Lab03/README.md
```

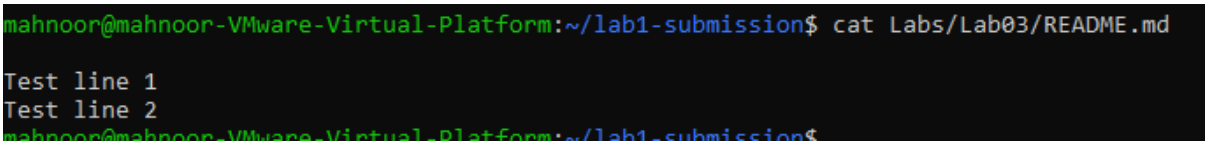


Figure 37: README.md still contains both lines after the soft reset

8. Because the changes were still staged, I committed them again.

```
git commit -m "Second Test commit"
```

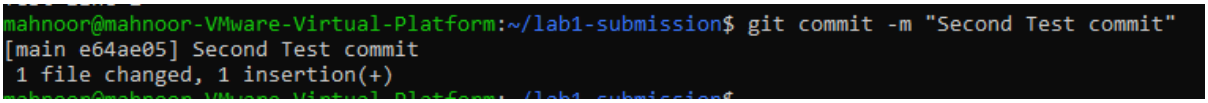


Figure 38: Recommitted after soft reset (e64ae05)

Hard reset

9. Now I did a hard reset to remove that commit and its changes.

```
git reset --hard HEAD~1
```

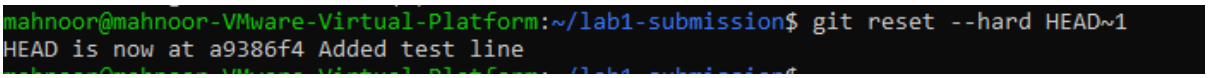


Figure 39: git reset --hard HEAD~1 – HEAD is now at a9386f4

10. The history shows “Added test line” as the latest commit.

```
git log --oneline
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git log --oneline
a9386f4 (HEAD -> main) Added test line
9df4dad (origin/main) Removed tracked textfiles directory
98ce603 Added .gitignore to ignore textfiles directory
e986073 Added textfiles directory with three files
d38a63d Local conflicting change
19f2c16 Update README.md
0c0e72b Resolved merge conflict in README.md
5aa0204 Local update to README
6bbba2a Update README.md
d556464 Updated directory structure according to requirements
7fdd998 Added screenshots folder and actual PDF
a3d1e8e Added labs structure and files
296b521 initial commit for lab 01
```

Figure 40: History after hard reset

11. In the file only “Test line 1” is left; “Test line 2” was discarded.

```
cat Labs/Lab03/README.md
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ cat Labs/Lab03/README.md

Test line 1
```

Figure 41: README.md after the hard reset contains only Test line 1

What I learned

- A soft reset removes the commit but keeps my changes staged in the files.
- A hard reset removes the commit and also throws away the changes in the files.
- After any reset I check both git log and the file itself to see what was kept.

Task 7 – Amending the Last Commit

Sometimes I forget to include something in a commit. git commit --amend fixes the last commit instead of making a new one.

1. I made a small change and committed it with the message “Fix log message”.

```
echo "Small update" >> Labs/Lab03/README.md
git add .
git commit -m "Fix log message"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Small update" >> Labs/Lab03/README.md
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Fix log message"
[main 7e3a5d7] Fix log message
 1 file changed, 1 insertion(+)
```

Figure 42: Original commit “Fix log message” (7e3a5d7)

2. I then updated another file (temp.txt) which I had forgotten, staged it and amended the last commit. I also updated the message while doing so.

```
echo "Forgot this line" >> Labs/Lab03/temp.txt
git add .
git commit --amend -m "Fix log message with updated temp file"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Forgot this line" >> Labs/Lab03/temp.txt
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit --amend -m "Fix log message with updated temp file"
[main d917f40] Fix log message with updated temp file
 Date: Wed Oct 14 15:14:11 2026 +0500
 2 files changed, 2 insertions(+)
```

Figure 43: Amended commit now includes 2 files (d917f40)

What I learned

- git commit --amend changes the latest commit’s message and/or the files in it.
- The amended commit gets a new hash (7e3a5d7 became d917f40).
- It is best used before pushing; after a push it needs a force push.

Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

git revert undoes a commit by creating a new commit, so the history is not rewritten. This is the safe way to undo something that is already on the remote.

1. I added a line to temp.txt, committed and pushed it.

```
echo "temporary text" >> Labs/Lab03/temp.txt
git add .
git commit -m "Added temporary text file"
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "temporary text" >> Labs/Lab03/temp.txt
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Added temporary text file"
[main 4372f07] Added temporary text file
 1 file changed, 1 insertion(+)
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 2 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (16/16), 1.33 KiB | 194.00 KiB/s, done.
Total 16 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), completed with 2 local objects.
To https://github.com/mahnoor-cs6767/DevOps.git
   9df4dad..4372f07  main -> main
```

Figure 44: Commit 4372f07 pushed to GitHub

2. I looked at the log to find the hash, then reverted that commit.

```
git log --oneline
git revert 4372f07 --no-edit
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git log --oneline
4372f07 (HEAD -> main, origin/main) Added temporary text file
d917f40 Fix log message with updated temp file
a9386f4 Added test line
9df4dad Removed tracked textfiles directory
98ce603 Added .gitignore to ignore textfiles directory
e986073 Added textfiles directory with three files
d38a63d Local conflicting change
19f2c16 Update README.md
0c0e72b Resolved merge conflict in README.md
5aa0204 Local update to README
6bbba2a Update README.md
d556464 Updated directory structure according to requirements
7fdd998 Added screenshots folder and actual PDF
a3d1e8e Added labs structure and files
296b521 initial commit for lab 01
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git revert 4372f07 --no-edit
[main 793f973] Revert "Added temporary text file"
 Date: Wed Oct 14 15:17:55 2026 +0500
 1 file changed, 1 deletion(-)
```

Figure 45: Revert commit 793f973 created

3. I pushed the revert commit.

```
git push origin main
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 484 bytes | 96.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/mahnoor-cs6767/DevOps.git
   4372f07..793f973  main -> main
```

Figure 46: Revert pushed (4372f07..793f973)

What I learned

- git revert <hash> adds a new commit that cancels the changes of an earlier one.
- The old commit stays in history, so it is safe for shared branches.
- For remote branches I use revert, not reset.

Task 9 – Force Push (With Caution)

I did this task only on my own test branch, not on main.

1. I created a new branch named test-force.

```
git checkout -b test-force
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git checkout -b test-force
Switched to a new branch 'test-force'
```

Figure 47: New branch test-force

2. I made a small change and committed it.

```
echo "Testing force push" >> Labs/Lab03/temp.txt
git add .
git commit -m "Testing force push"
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ echo "Testing force push" >> Labs/Lab03/temp.txt
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git add .
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git commit -m "Testing force push"
[test-force dae344a] Testing force push
 1 file changed, 1 insertion(+)
```

Figure 48: Commit "Testing force push" (dae344a)

3. I pushed the new branch to GitHub.

```
git push origin test-force
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin test-force
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 468 bytes | 93.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'test-force' on GitHub by visiting:
remote:      https://github.com/mahnoor-cs6767/DevOps/pull/new/test-force
remote:
To https://github.com/mahnoor-cs6767/DevOps.git
 * [new branch]      test-force -> test-force
```

Figure 49: test-force pushed as a new branch

4. I removed that commit locally with a hard reset. Now my local branch is one commit behind the remote one.

```
git reset --hard HEAD~1
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git reset --hard HEAD~1
HEAD is now at 793f973 Revert "Added temporary text file"
```

Figure 50: Hard reset – HEAD is now at 793f973

5. A normal push was rejected as non-fast-forward.

```
git push origin test-force
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin test-force
To https://github.com/mahnoor-cs6767/DevOps.git
 ! [rejected]        test-force -> test-force (non-fast-forward)
error: failed to push some refs to 'https://github.com/mahnoor-cs6767/DevOps.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Figure 51: Normal push rejected (non-fast-forward)

6. I then used a force push to overwrite the remote branch with my local history.

```
git push origin test-force --force
```

```
mahnoor@mahnoor-VMware-Virtual-Platform:~/lab1-submission$ git push origin test-force --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/mahnoor-cs6767/DevOps.git
 + dae344a...793f973 test-force -> test-force (forced update)
```

Figure 52: Forced update: dae344a...793f973

What I learned

- git push --force rewrites the remote history with my local history.
- It should only be used on my own branches that nobody else uses, never on main.
- I should always double check before using it.

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

In this task I ran a real application, Gitea (a self-hosted Git service), inside a GitHub Codespace using Docker Compose.

1. I forked the Gitea tutorial repository into my GitHub account (with "Copy the main branch only" unchecked so that the codespace branch is included).

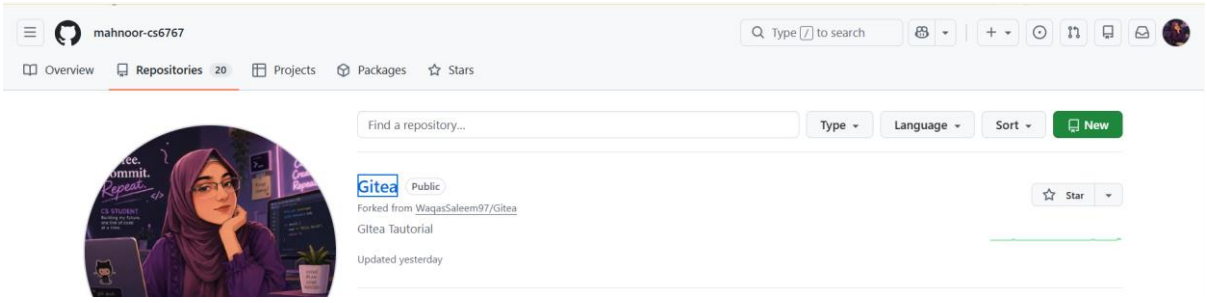


Figure 53: Forked Gitea repository in my GitHub account

2. I opened the fork in a codespace using `Code > Codespaces > Create codespace on main`.

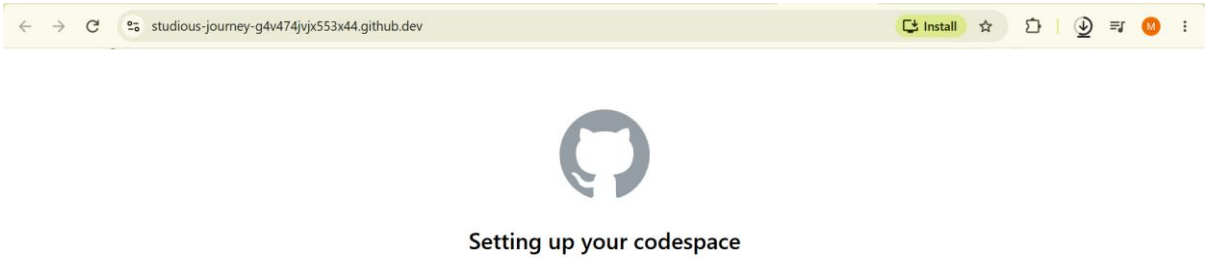


Figure 54: Codespace being set up

3. In the codespace terminal I switched to the codespace branch and started the containers.

```
git checkout codespace
docker compose up -d
```



Figure 55: gitea and gitea_db containers started

4. I forwarded port 3000, made it public and opened it in the browser. The Gitea Initial Configuration page appeared (PostgreSQL database settings).

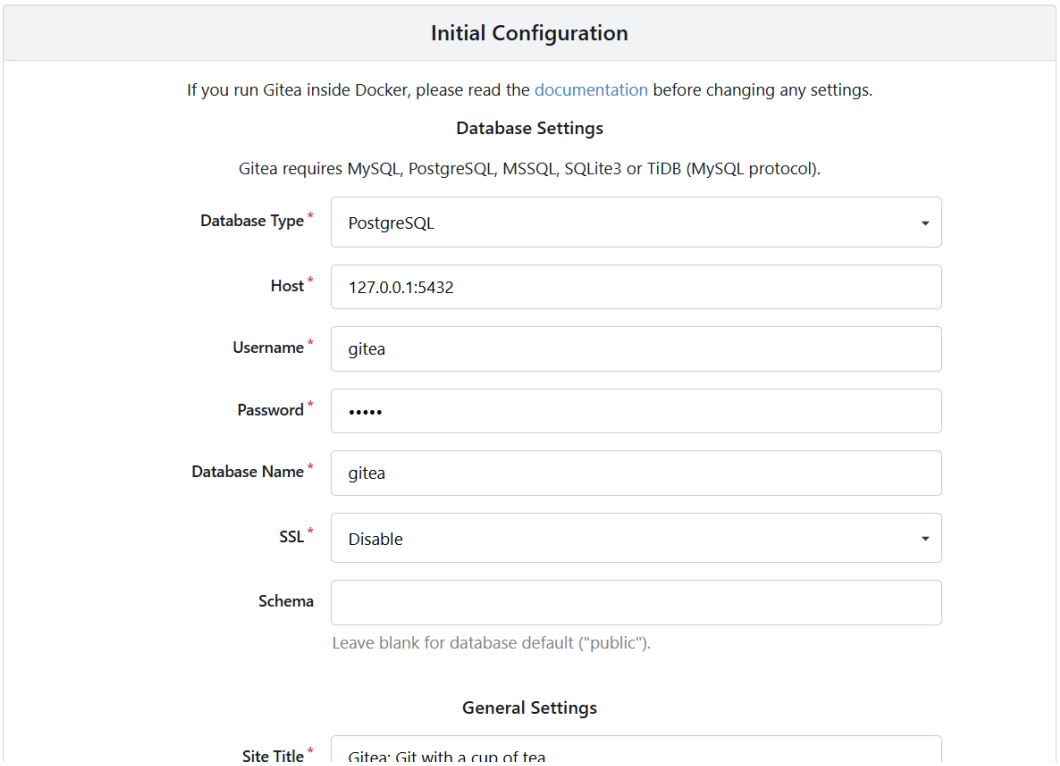


Figure 56: Gitea installation page

5. I filled in the administrator account details (username admin) and submitted the form.

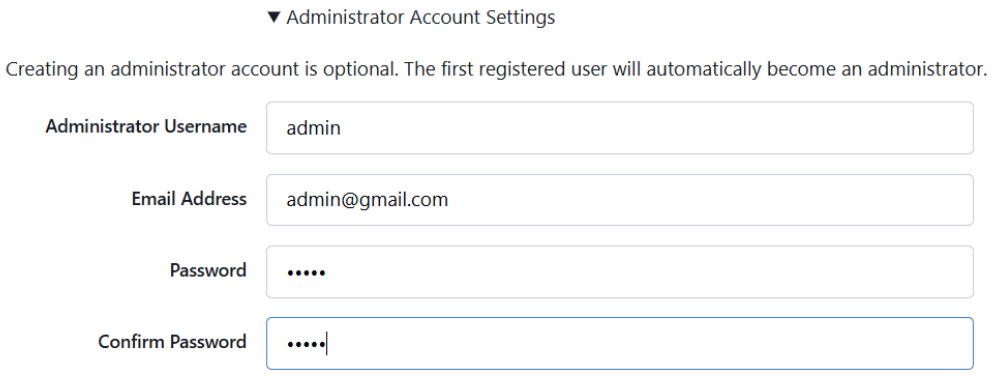


Figure 57: Administrator account settings on the install page

6. After installation I logged in as admin and reached the dashboard.

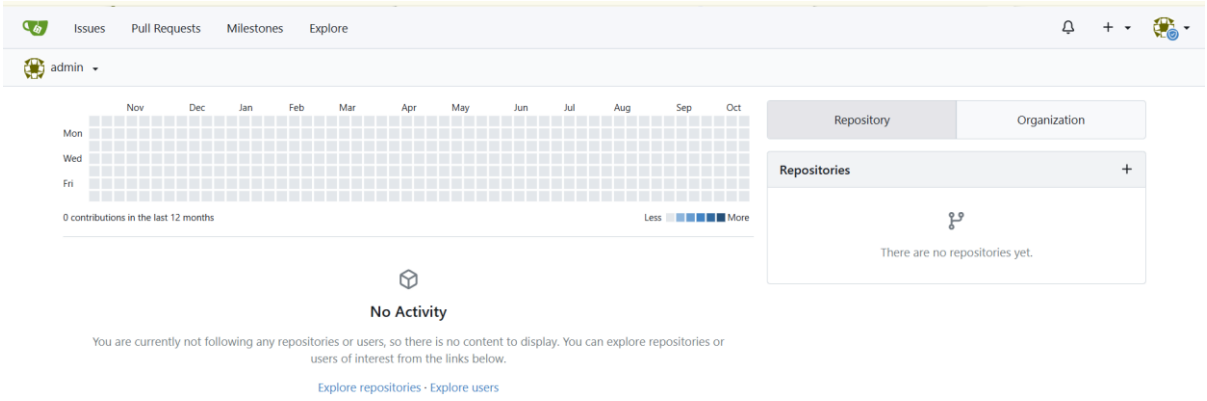


Figure 58: Gitea dashboard after logging in

7. I created a new repository in Gitea named DevOps-Lab03.

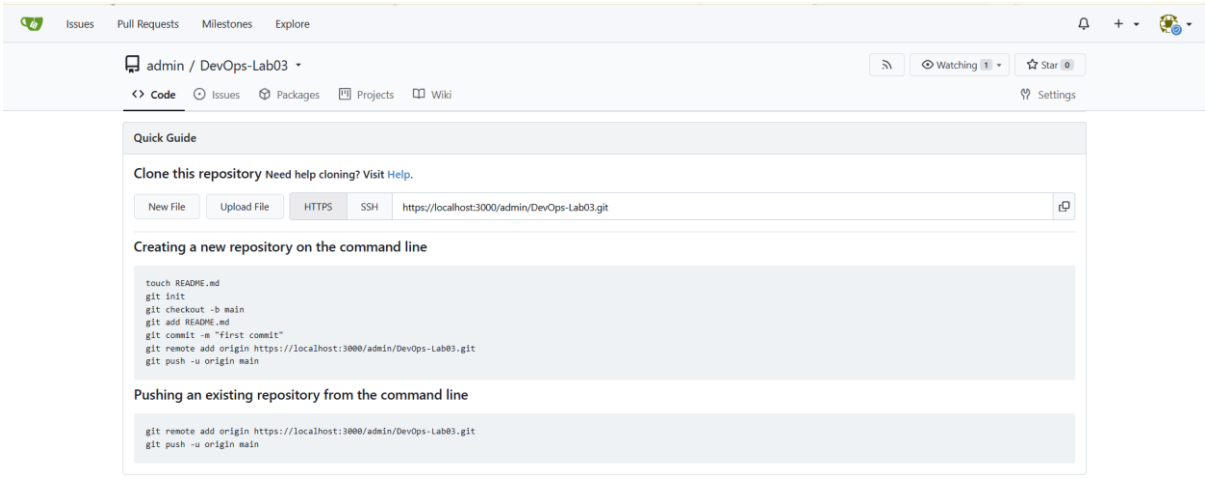


Figure 59: New repository admin/DevOps-Lab03 in Gitea

What I learned

- I forked a project and ran a real application with Docker Compose in Codespaces.
- I installed and configured Gitea from scratch and logged in as an admin.
- I created a repository in Gitea, similar to how I do it on GitHub.

Task 11 – Creating a GitHub Pages Portfolio Site

The last task was to publish a personal CV/portfolio page using GitHub Pages.

1. I created a public repository named mahnoor-cs6767.github.io.

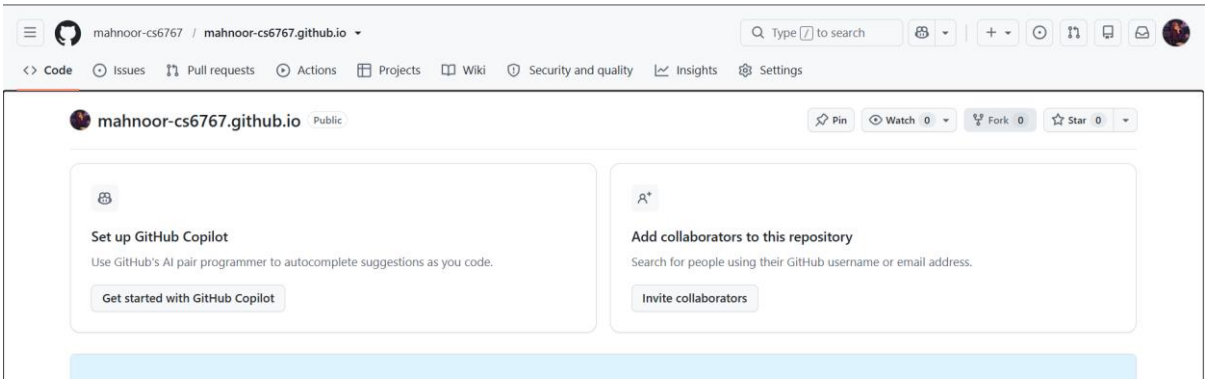


Figure 60: New public repository mahnoor-cs6767.github.io

2. I prepared the static site (index.html and style.css) in a local folder in VS Code.

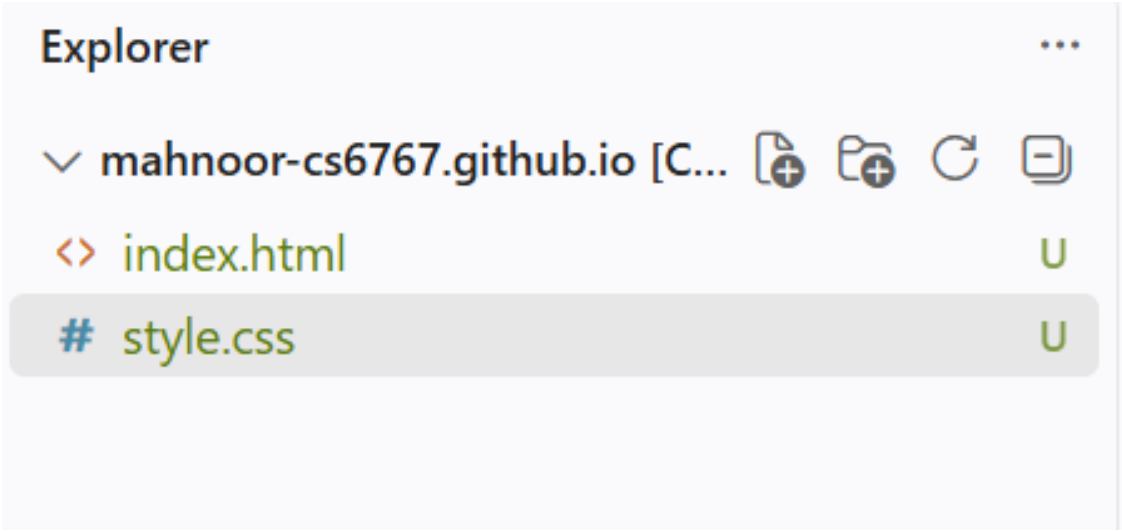


Figure 61: index.html and style.css in the local project folder

3. I pushed the files to GitHub.

```
git add .
git commit -m "Add portfolio site for GitHub Pages"
git push -u origin main
```

```
@mahnoor-cs6767 →/workspaces/mahnoor-cs6767.github.io (main) $ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.63 KiB | 2.63 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/mahnoor-cs6767/mahnoor-cs6767.github.io.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 62: Site files pushed to main

4. In Settings > Pages I confirmed that the site is deployed from the main branch (root folder) and live at https://mahnoor-cs6767.github.io/.

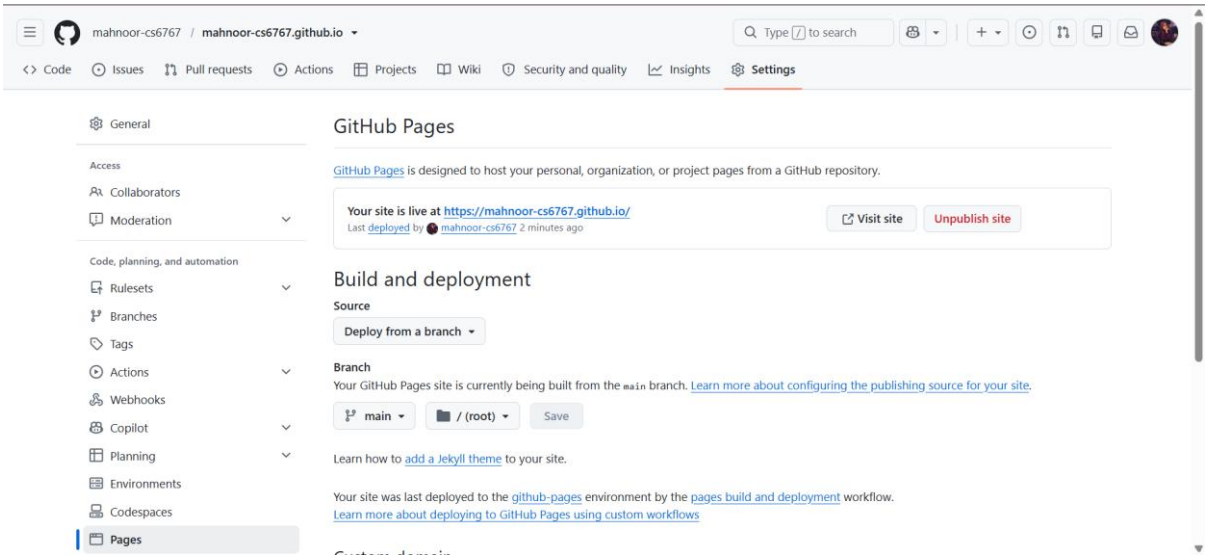


Figure 63: GitHub Pages settings showing the live site

5. I opened the link in the browser to check the live site.

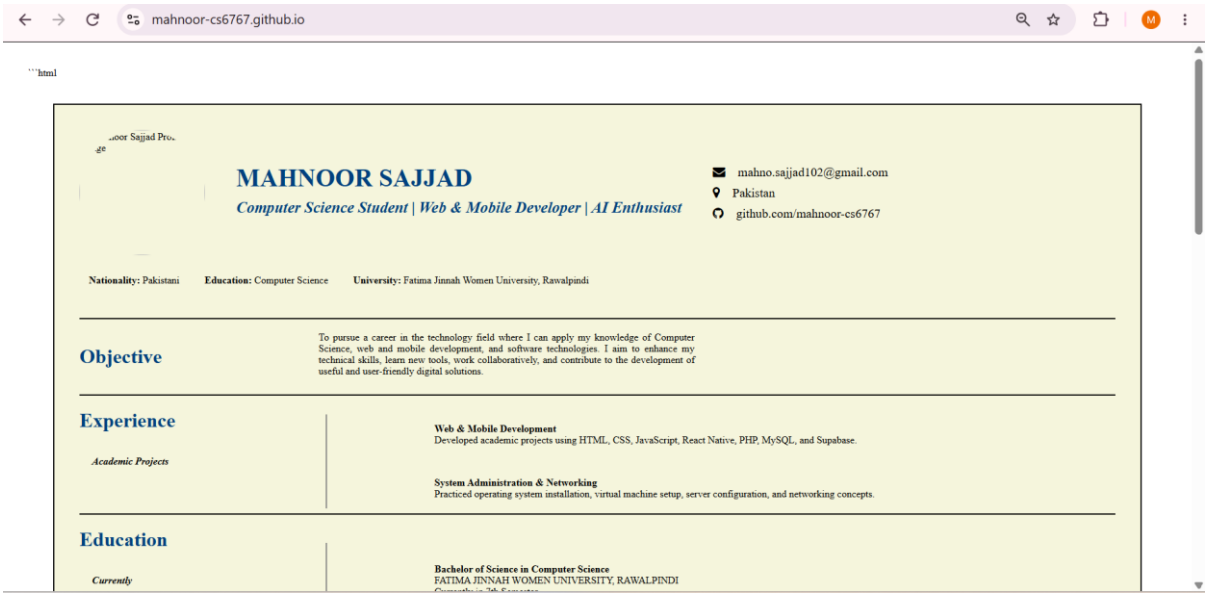


Figure 64: Live portfolio site at mahnoor-cs6767.github.io

What I learned

- A GitHub Pages repository must be named <username>.github.io.
- Pushing HTML and CSS files to main is enough to publish a static site.
- The Pages settings show the live URL once the deployment is finished.

Exam Evaluation Questions

The four practical exam questions use the same Git operations I performed in the tasks above, so my evidence for them is the work shown in those tasks:

Q1 – Local vs Remote Conflict Resolution

Done in Task 1: remote edit on GitHub, local commit, rejected push (Figures 1–3), merge resolution with git pull and push (Figures 4–5), then git pull --rebase and push (Figures 6–7).

Q2 – Manual Merge Conflict Handling

Done in Task 2: conflicting remote and local edits, rejected push, git pull --rebase triggering the conflict, manual resolution in nano, git add and git rebase --continue, then push.

Q3 – Managing Ignored and Tracked Files

Done in Task 3: folder created and pushed, added to .gitignore, removed from tracking with git rm -r --cached, and verified as removed on GitHub.

Q4 – Commit History Manipulation and Recovery

Done in Task 6: two commits, git log, soft reset, recommit, and hard reset, each with the log and file contents checked.

Summary

In this lab I learned how Git handles situations where history and files get out of sync, and how to fix them in a safe way. The main points I take from it are:

- git pull --no-rebase merges, git pull --rebase keeps history linear, and conflicts must be fixed by hand.
- .gitignore stops new files from being tracked, but git rm --cached is needed for files that are already tracked.
- git stash keeps unfinished work safe while I switch branches.
- A soft reset keeps my changes, a hard reset discards them, and git commit --amend fixes the last commit.
- git revert is the safe undo for a shared branch, while git push --force rewrites history and must only be used on my own branches.
- I also ran Gitea with Docker Compose in a Codespace and published a portfolio site on GitHub Pages.

Submission

GitHub repository: **https://github.com/mahnoor-cs6767/DevOps**

Submission folder: Labs/Lab03

Screenshots folder: Labs/Lab03/screenshots